

But in this article, we'll focus more on the bulk transfer between an Android app and accessory host, which is known as custom accessory mode.

Re-enumeration of the device after initialisation

Upon initialising the device as mentioned above, it will re-enumerate in the accessory mode with a unified configuration, in which interface 0 comes with two end points meant for bulk transfer operations. If the device is in audio or ADB mode, a few more interfaces will be detected. The re-enumerated configuration can come with one of the following vendor ID-product ID combinations, depending on the initialisation (accessory, audio support) and ADB mode of the device.

0x18D1:0x2D00	Accessory only
0x18D1:0x2D01	Accessory + ADB
0x18D1:0x2D02	Audio only
0x18D1:0x2D03	Audio + ADB
0x18D1:0x2D04	Audio + Accessory
0x18D1:0x2D05	Audio + Accessory + ADB

Then the device will try to launch suitable applications that match with the manufacturer, model, and the version identity strings specified as intent filters in the manifest files of the Android app. If no suitable application is installed, a pop-up will be shown with a specific URL (if any) pointing to the APK file or with details of the application.

Device side API and code flow

Here is a brief discussion on APIs and codeflow on the Android side with a sample code from Google I/O 2012, which may be helpful to develop new apps or customise existing ones.

Once ADK initialises the device, the USB manager tries to match the sent identity strings with intent-filter content as mentioned in the manifest resource of an app. Upon verification and user permissions, the application's activity starts with `USB_ACCESSORY_ATTACHED` intent; and similarly, `USB_ACCESSORY_DETACHED` intent will be sent when ADK is disconnected.

```
import android.hardware.usb.UsbAccessory;
import android.hardware.usb.UsbManager;
mUsbManager = (UsbManager) getSystemService(Context.
USB_SERVICE);
UsbAccessory acc; acc=(UsbAccessory)intent.
getParcelableExtra( UsbManager.EXTRA_ACCESSORY );
if (!mUsbManager.hasPermission(acc)) return;
ParcelFileDescriptor mFD = mUsbManager.
openAccessory(acc);
if (mFD != null) {
    FileDescriptor fd = mFD.getFileDescriptor();
    mIS = new FileInputStream(fd);
    mOS = new FileOutputStream(fd);
}
```

Once we obtain references for input and output streams as illustrated above, the rest of the flow depends on the communication between the app and ADK.

Sample Android applications

Here is a listing of some applications designed to work with various embedded targets as ADK, once your Android device is turned into open accessory mode. You may use one of these applications for initial work until you start designing your own Android apps like me, or customise those apps for which source code is available.

From the Google Play store

- ADK 2012 by Google, https://dl-s1.google.com/android/adk/adk_release_20120606.zip
- Standard Android ADK Demo Kit (ADK 2011) by DIY Phone Gadgets
- Basic accessory demo from Microchip

From other sources

- TI-ADK Demo from <https://code.google.com/p/rowboat/>
- http://www.embeddedartists.com/products/app/aaa_kit.php
- Few apps from <http://www.ftdichip.com/Android.htm>



Note: Some of these applications not starting or stop responding in few devices, some debugging and fixing may be required.

A case study—Linux as ADK

To turn a Linux machine to an ADK, I have written a skeleton driver code, hosted at github.com/rajeshsola/adk-driver-skeleton based on Gary Bisson's Linux ADK code. One can write user space apps on top of this driver or customise the skeleton code for specific needs.

As a case study, I've chosen the SimpleAccessory app available from nexus-computing.ch to test with Linux as the ADK. By changing the simple slider in this app, you can change some properties of ADK like volume, brightness, etc.

Load the driver `adk-aaa-skeleton.ko` using `insmod` or `modprobe`. Preferably, you can use module params `my_device_vid, my_device_pid` with vendor ID and product ID of the device in order to probe the device on detection, or you can use the necessary udev rules to load the module on connecting the device. Stop any storage related services and unload other modules that prevent association between this driver and the device, before connecting.

Once the device has been probed, sysfs operations are enabled under `/sys/kernel/adk_linux/aaa_init/` for initialisation purposes; so, let's use simple commands to initialise the device.